15,475,996 members Sign in

articles Q&A forums stuff lounge ? Search for articles, questions,

File Explorer Clone with Tabs and Parallel File Search Tool



Wade Harvey
21 Apr 2018 CPOL

Rate me: 4.84/5 (15 votes)

This article shows how to use Winforms, WPF and C# to create a File Explorer Clone with Tabs and an extremely fast parallel file search utility.

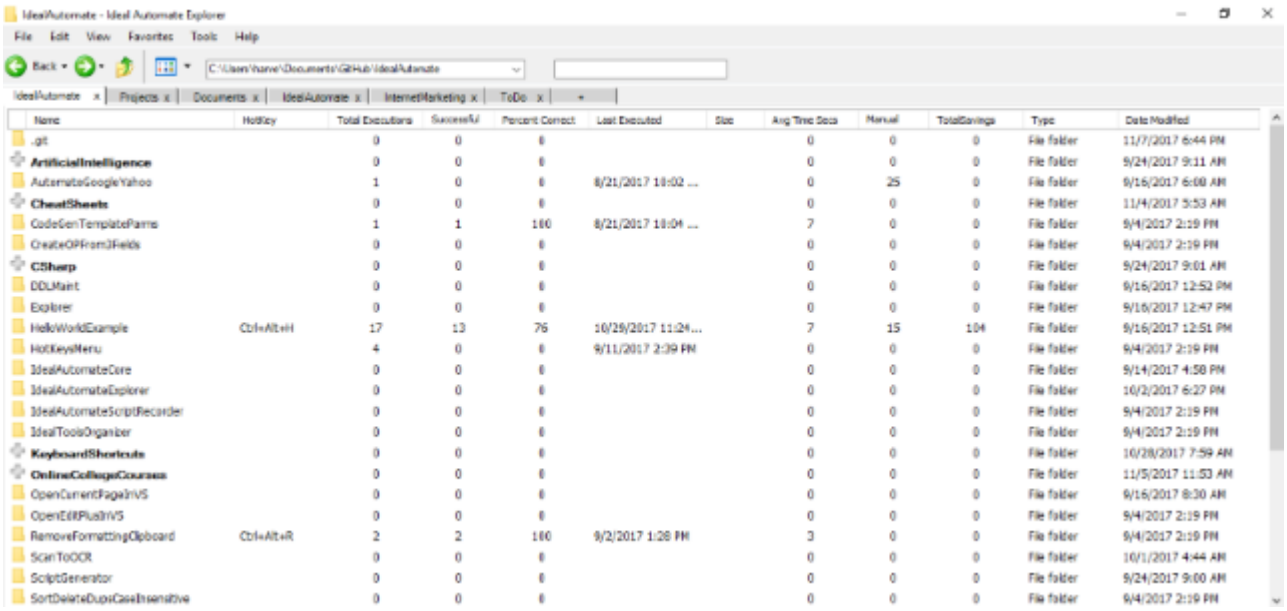
Download source - 86 MB

Download latest executable for File Explorer Clone without source - 11 MB

Introduction

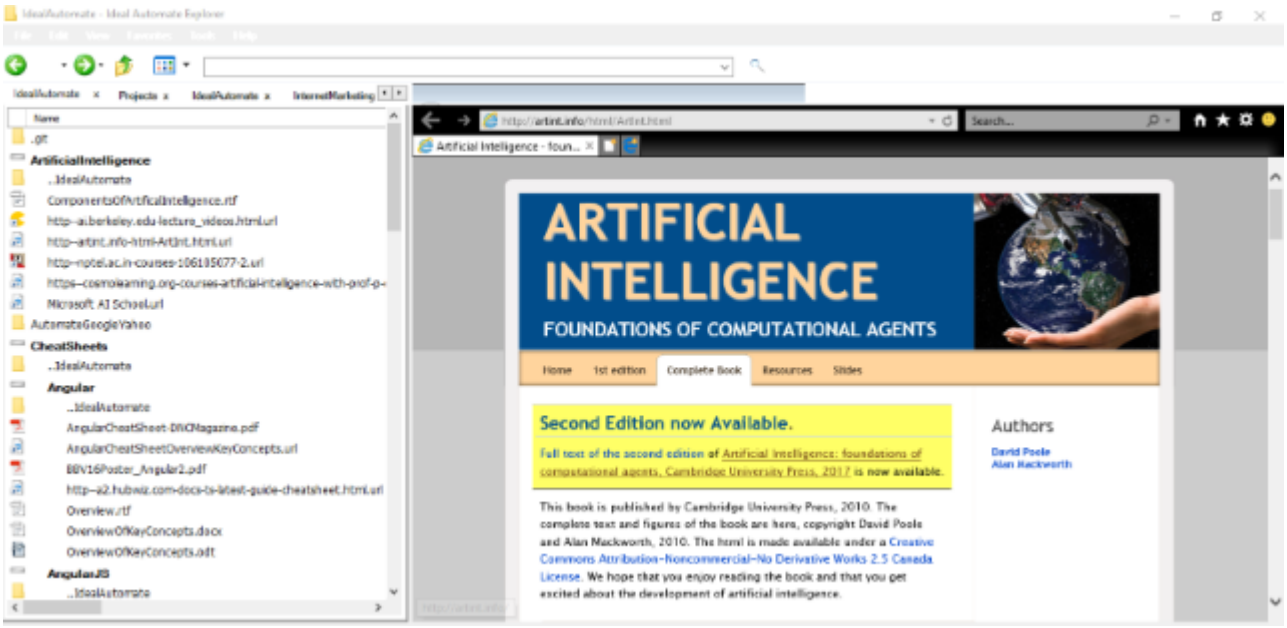
This article shows how to use Winforms, WPF, C# to create a File Explorer Clone with Tabs that include an extremely fast parallel file search utility. I use this File Explorer to create, and organize, repositories of three types of things: C# WPF projects that automate redundant computer tasks; Knowledge Bases; and ToDoLists.

Here is a screenshot of the File Explorer Clone in List View:



(Click to enlarge)

Here is a screenshot of the File Explorer Clone in Details View:



(Click to enlarge)

Background

My original goal was to automate problem-solving itself, but in the process of trying to do that, I ended up creating a clone of File Explorer. I wanted to automate problem-solving because I saw a general pattern and because most of my programming time is spent performing problem-solving tasks.

Here is the general pattern, or algorithm, that can be used to solve any problem:



A problem arises when there is a "gap" between the **Actual** and the **Ideal**.

1. The first step to solving the problem is to compare the Actual to the Ideal and find the **differences**.
2. Once the differences have been identified, we need to translate those differences into Actions that we know how to perform.
3. Once the Differences have been translated into Actions, we perform those actions on the Actual in order to transform it into the Ideal.

Artificial Intelligence uses the above steps to solve problems. AI has a knowledge base of actions it knows how to perform, and it uses those actions to remove the differences between the way something actually is and the way it ideally should be.

A **knowledge base** can be used to store the translations between our understanding of something (represented in human language) and the computers understanding (scripts written in computer language). The knowledge base serves as a **key-value pair dictionary**, with the human language as the "key" and the script in computer language as the ultimate "value."

In the same way that Google maps finds the shortest distance between two locations, AI uses the knowledge base to determine the fewest actions needed to be performed in order to transform the actual problem state into the ideal solution state.

What I originally wanted to do was to create an application that would do the above problem-solving steps for me, but this proved to be too complicated. As a result, I settled for writing many little C# WPF applications that would automate little things that I knew how to do. After writing quite a few of these little C# WPF applications, I realized that I needed an easy way to put these little action scripts into some sort of knowledge base and launch them with hotkeys.

This is what led me to create this clone of File Explorer that has tabs. I decided to add tabs to my File Explorer clone because I do not like the way File Explorer requires you to open up a different instance of File Explorer every time you want to look at a different folder. The tabs also provide the benefit of allowing you to save where you are in multiple file explorers so that you can come back to it later.

In order to categorize information in a knowledge base, you need to be able to create categories and subcategories in addition to just having folders. The difference between categories/subcategories and regular folders is that categories/subcategories allow you to persist what you have expanded/closed when you leave the folder structure while folders do not.

I soon realized that I often like to edit or view files on the fly without exiting the file explorer clone, and this is what led me to create the preview mode.

(Click to enlarge)

Search is another important feature in any knowledge base, so I added a search icon that does a parallel search in the background when clicked. Search allows you to find all instances of `string` or regular expression in text, word, or open office files included within all of the subfolders of a parent folder. You can use a hotkey to navigate to the particular line in the result in a file or to open a new tab in the File Explorer clone that points to the immediate parent folder of the file that contains the text that was found.

(Click to Enlarge)

In order to add hotkeys, metadata, and other functionality to the folders/files, I added a context menu when the user right-clicks on the folder/file.

Using the Code

The easiest way to get started is to just download the executable. However, if you download the source code, you will find that it contains a number of solutions and projects. These solutions and projects demonstrate several of the things that you can automate and use the file explorer clone to do. Since this article is only focusing on the file explorer clone, the solution that you will want to build is the one called `IdealAutomateExplorer`.

The `IdealAutomateExplorer` solution consists of three projects: `dgFilterPopup`; `IdealAutomateExplorer`; and `IdealSqlTracer`.

`dgFilterPopup` is an excellent library that allows one to add filtering capabilities to the `datagridview`. I did not write this project, and I am greatly indebted to Vincenzo Rossi for sharing information about this library in his superb CodeProject article. Here is a screenshot of the features that this library adds to the File Explorer clone:

The second project in the `IdealAutomateExplorer` solution is the `IdealAutomateExplorer` project. The main page in this project is the *ExplorerView.cs* page. This page contains the `tabControl`. Inside each `tabPage` in the `tabControl`, there is a `splitContainer`. `Panel1` of the `splitContainer` is used to hold the `datagridview` for the folder of the current tab. `Panel2` is used in preview mode to display the file that has been selected in the `datagridview`. Wordpad is used to open any Word type documents because Wordpad is a free tool that should be on everyone's computer whereas Word is not.

The third project in the solution is `IdealSqlTracer`. `IdealSqlTracer` can be accessed from the Tools/Sql/IdealSqlTracer menu option. `IdealSqlTracer` provides you with an alternative to `SqlProfiler` that allows you capture all of the SQL behind any desktop or web application. One nice feature of `IdealSqlTracer` is that it takes the captured SQL and formats it so that it is easy to read. There is plenty of documentation in the help screens to learn more about this powerful tool.

Points of Interest

There are two places where information is stored for this application to allow it to be persisted when the application is closed.

The first place is in the file directory structure itself. Whenever you add a category or a subcategory, information about whether that category/subcategory is expanded is placed in a folder that has the name... `IdealAutomate`.

The second place information is kept is in the roaming directory. On my computer, the name of this folder is *C:\Users\harve\AppData\Roaming\IdealAutomate\IdealAutomateExplorer*, but the name on your computer will be slightly different. There are methods called `SetValueByKey` and `GetValueByKey` that are used to put information into this roaming folder and to retrieve it from there.

History

- 4-21-2018: Originally submitted for publication

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By

Wade Harvey

Software Developer IdealProgrammer.com

United States

My name is Wade Harvey. I have been in the programming field for thirty years. I have a blog at http://idealprogrammer.com and a website at http://idealautomate.com My github repositories are located at: https://github.com/harvey007y In addition to enjoying programming, I also have a background in psychology and philosophy.

One of the things that I have learned from philosophy is that what makes things good is how well they

match up to their ideal. One ethicist I studied in philosophy was named Robert Hartman. Hartman applied Field Theory to values.

In a field, there are two poles. This situation of there being two poles is often referred to as being "bipolar." In the field of values, the two poles are the actual object and the ideal.

According to Hartman, the tension that we feel between the actual and the ideal object is similar to the tension that exists between the two poles of a magnet. This tension exists not only between actual objects and ideal ones, but also between our actual self and our ideal one. When we take action to transform the actual into the ideal, we are able to reduce the tension that exists between the two.

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

Many images are missing
Mou_kol 11-Jun-18 20:28

Like looking in a mirror 'cept you're a shrink
Mitchster 25-May-18 5:43

Re: Like looking in a mirror 'cept you're a shrink
Wade Harvey 9-Jul-18 2:53

Spam
Member 13797003 24-Apr-18 21:09

Many code
LightTempler 22-Apr-18 9:12

Re: Many code
Member 2443306 22-Apr-18 12:34

Re: Many code

TheNexpert 11-May-18 0:47

Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2018 by Wade Harvey
Everything else Copyright © CodeProject, 1999-2022

Web04 2.8:2022-10-01:1